Green University of Bangladesh

Department of Computer Science and Engineering (CSE)

Semester: (Summer, Year: 2026), B.Sc. in CSE (Day)

Minesweeper

Project Report

Course Title: Web Prgramming Lab
Course Code: CSE 302
Section: 242_D1

Students Details

Name	ID
Mohammad Shahria Faysal	242002056

Submission Date: [25-8-2026]

Course Teacher's Name: **Ms. Umme Habiba**

[For teachers use only: Don't write anything inside this box]

Lab Project Status	
Marks:	Signature:
Comments:	Date:

Contents

List of Figures

List of Tables

Abstract

This report documents the design, implementation, and evaluation of a web-based Minesweeper game built as a full-stack, single-page application. The frontend is implemented in React with Vite and communicates with a procedural PHP backend over a JSON-based API, using PDO prepared statements against a MySQL database for persistent storage. Unlike traditional Minesweeper implementations, the game does not track a "clear the board" win condition; instead, it is played as a timed survival mode in which the player attempts to reveal as many safe cells as possible before either clicking a mine or the two-minute countdown expires. The game engine – including deterministic-feeling first-click safety, an iterative flood-fill reveal algorithm, chording, a random-cell hint system, and a single-use shield ability – runs entirely on the client. The backend does not participate in gameplay; its role is limited to user authentication (registration, login, and session-based authorization) and to independently revalidating and persisting the score of a completed game, so that a client cannot submit a fabricated result. This report covers the system's requirements, the client-side game algorithms in detail, the overall software architecture, the database schema, the implementation of the frontend and backend, the security mechanisms applied (including server-side score recomputation and duplicate-submission prevention), the major user-facing features, the testing strategy used to validate the system, and the project's known limitations and possible future improvements.

Keywords: Minesweeper, Web Application, React, Vite, PHP, MySQL, Client-Side Game Logic, Session Authentication, Score Validation.

Chapter 1

Introduction

1.1 Background

Minesweeper is a classic single-player logic puzzle in which a player must uncover the safe cells of a grid while avoiding hidden mines, using numeric clues that indicate how many mines are adjacent to a revealed cell. As a well-understood game with clear, self-contained rules, it is a natural candidate for demonstrating full-stack web development skills: it requires a non-trivial client-side algorithm (board generation and cascading reveal), a persistent notion of user identity and history, and a backend capable of storing and ranking results in a way that resists tampering by the client.

1.2 Problem Statement

Implementing an online Minesweeper game that supports user accounts, score tracking, and a public leaderboard raises a specific integrity problem: if the game logic and the final score are both computed on the client, a malicious user could submit an arbitrarily inflated score directly to the backend without ever actually playing a valid game. A trustworthy implementation must therefore separate the concerns of *gameplay* (which can reasonably run in the browser for responsiveness) from *score validation* (which must be authoritative on the server).

1.3 Motivation

The motivation behind this project was to build a complete, working Minesweeper web application that goes beyond a purely client-side game by adding user accounts, persistent game history, a global leaderboard, and personal statistics – while addressing the score-integrity problem described above through explicit server-side validation. The project also served as a practical exercise in structuring a React single-page application (contexts, hooks, and a services layer) alongside a lightweight, framework-free PHP JSON API.

1.4 Proposed Solution

The application separates the game into two layers. On the client, a pure functional module (`minesweeper.js`) implements board generation with first-click safety, an iterative flood-fill reveal, flag toggling, chording, a limited hint system, and score calculation. A custom React hook (`useMinesweeper`) drives this module and manages the game's timer and state transitions. On the server, PHP endpoints handle registration, login, and logout using PHP sessions, and a dedicated `save-score.php` endpoint independently recomputes the score from the submitted raw metrics (cells revealed, time taken, hints used) rather than trusting a client-supplied score value, before persisting the result to MySQL.

1.5 Project Objectives

The specific objectives of this project were to:

- Implement a client-side Minesweeper game engine with first-click safety, flood-fill reveal, flagging, chording, hints, and a shield ability, within a fixed-time survival mode.
- Build a user account system with registration, login, logout, and session-based authorization.
- Persist completed game results to a relational database and expose them through a personal history view, a personal statistics dashboard, and a global leaderboard.
- Ensure that submitted scores cannot be trusted at face value from the client, by recomputing and bounding them on the server before storage.
- Apply standard web security practices, including password hashing, session-fixation mitigation, restrictive CORS configuration, and parameterized SQL queries.

1.6 Scope

The application supports exactly two difficulty levels – Beginner (9×9 grid, 10 mines) and Advanced (16×16 grid, 40 mines) – and a single game mode: a 120-second survival mode in which the game ends only when a mine is revealed or the timer reaches zero. There is no "you cleared the board" win condition. The system supports a single user role (registered player); there is no administrator role. Features such as password reset, real-time multiplayer, and push notifications are not implemented and are discussed as possible future work in Chapter 13.

Chapter 2

System Overview

2.1 Application Overview

The application is a single-page application (SPA) built with React and Vite, communicating with a PHP backend through a JSON HTTP API. The browser never performs a full page reload while navigating between the game, the dashboard, the leaderboard, and the history page; instead, React Router switches between page components while shared state (authentication and theme) is provided through React Context.

2.2 Target Users

The system supports a single user role:

- **Registered Player** – creates an account, plays timed Minesweeper games at one of two difficulties, and can view a personal dashboard of statistics, a full history of past games, and a global leaderboard of top scores across all players.

There is no administrator or moderator role; every authenticated user has identical privileges over their own data.

2.3 System Workflow

At a high level, a player's session follows the flow shown in Figure 2.1: the player registers or logs in, is taken to a dashboard, starts a game at a chosen difficulty, plays until the game ends (by mine or timeout), and the result is automatically submitted to the backend and reflected in the player's history, statistics, and the global leaderboard.

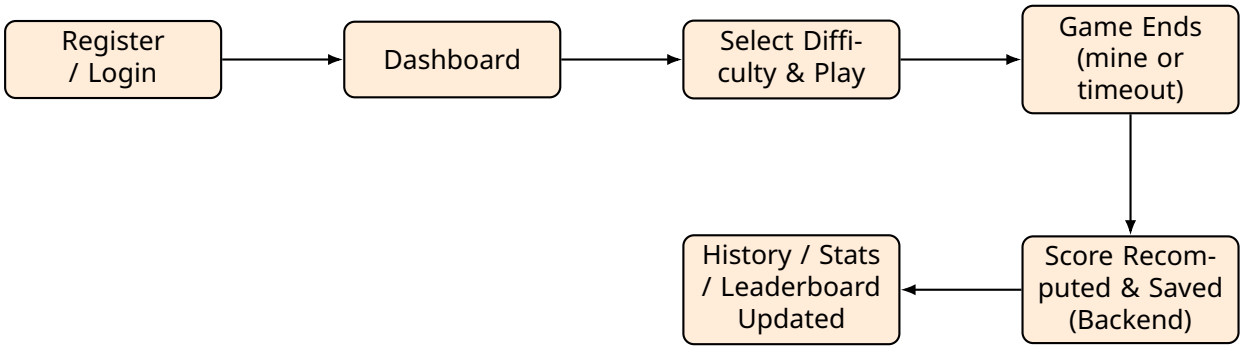


Figure 2.1: High-level player workflow.

2.4 Main Modules

The application is organized into the following functional areas, described in detail in later chapters:

- **Authentication Module** – registration, login, and logout endpoints backed by PHP sessions (`register.php`, `login.php`, `logout.php`).

- **Game Engine Module** – the pure functional board logic in `minesweeper.js` and the stateful `useMinesweeper` hook that drives it.
- **Score Persistence Module** – the `save-score.php` endpoint, which validates and recomputes a submitted result before writing it to the `game_results` table.
- **Statistics and History Module** – the `leaderboard.php`, `history.php`, and `stats.php` endpoints and their corresponding React pages.

2.5 Screenshots

Figures 2.2 and 2.3 show the authentication screens; Figure 2.4 shows the authenticated dashboard.

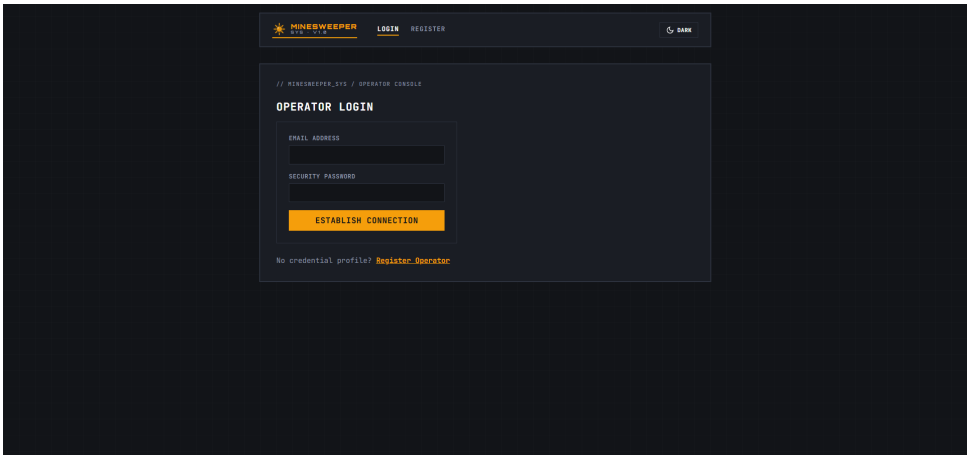


Figure 2.2: Operator login page.

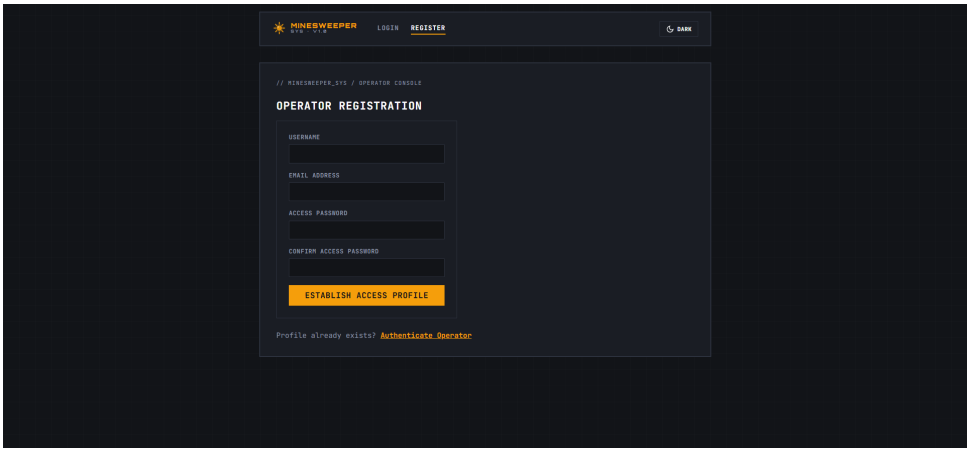


Figure 2.3: Operator registration page.

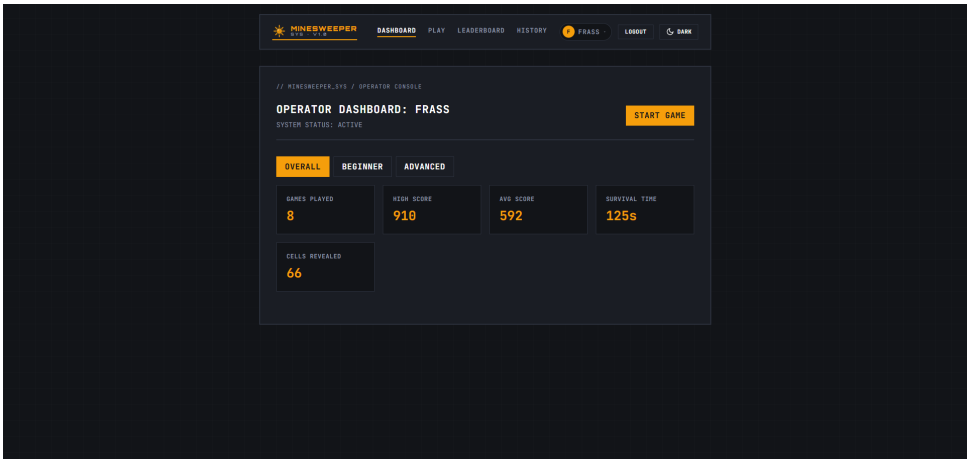


Figure 2.4: Player dashboard showing combat statistics and quick-access links.

Chapter 3

Requirements Analysis

3.1 Functional Requirements

The functional requirements below are derived directly from the behaviour implemented in the source code.

1. The system shall allow a new user to register with a username (3–50 characters), an email address, and a password of at least 6 characters, confirmed twice.
2. The system shall reject registration when the supplied email is not a valid email address, or when the username or email is already in use.
3. The system shall hash passwords with `password_hash()` before storing them, and never store plaintext passwords.
4. The system shall allow a registered user to log in with their email and password, verified with `password_verify()`, and shall regenerate the session ID on successful login.
5. The system shall allow a logged-in user to log out, which clears and destroys the server-side session.
6. The system shall allow a logged-in user to start a game at one of two difficulties: Beginner (9×9, 10 mines) or Advanced (16×16, 40 mines).
7. The system shall guarantee that the first cell revealed by the player, and all of its immediate neighbours, never contain a mine.
8. The system shall reveal a cascade of adjacent zero-value cells automatically when a zero-adjacency cell is revealed (flood reveal).
9. The system shall allow the player to flag and unflag unrevealed cells, up to the total mine count for the active difficulty.
10. The system shall allow the player to chord on a revealed numbered cell when the number of adjacent flags equals its adjacent mine count, automatically revealing the remaining unflagged neighbours.
11. The system shall allow the player up to 3 hints per game, each highlighting a random unrevealed, non-mine cell for 3 seconds.
12. The system shall grant the player exactly one shield per game, which, once activated, prevents the next mine reveal from ending the game.
13. The system shall end the game when the player reveals a mine (without an active shield) or when the 120-second timer reaches zero, and shall reveal all mines at that point.
14. The system shall automatically submit the completed game's metrics (difficulty, time taken, cells revealed, hints used) to the backend once, and shall not re-submit the same completed game.
15. The system shall recompute the score on the server from the submitted metrics rather than accept a client-supplied score, and shall reject metric values that fall outside the valid range for the given difficulty.
16. The system shall reject a duplicate submission of the same result signature (user, difficulty, time, cells, hints) received within 10 seconds of the previous one.
17. The system shall display, for a logged-in user: personal statistics (games played, high score, average score, longest survival, most cells revealed), a history of up to 200 past games, and a global top-10 leaderboard, each filterable by difficulty.

3.2 Non-Functional Requirements

- **Security** – passwords must never be stored in plaintext; all SQL queries that include user input must use PDO prepared statements; session cookies must be HTTP-only; cross-origin requests must be restricted to an explicit allow-list of origins.
- **Data Integrity** – the `users` table must guarantee unique usernames and unique email addresses; every stored game result must be linked to a valid user via a foreign key.
- **Usability** – the interface should allow a player to start a game, understand their remaining time, mines, flags, and score, and review a completed game's outcome, without external instructions.
- **Maintainability** – difficulty parameters (grid size, mine count) should be defined in a single, centralized configuration object on the frontend rather than duplicated across components.

3.3 User Requirements

A player needs an account, a way to start and play a timed game at a chosen difficulty, visible feedback on their current score and remaining time and mines, a way to review their past results, and a way to compare their best scores against other players.

3.4 Requirements Traceability

Table 3.1 maps a representative subset of the functional requirements above to the test cases used to validate them in Chapter 10.

Table 3.1: Representative requirements-to-test-case mapping.

Req. #	Requirement	Test ID
FR-3	Password hashing on registration	T-2
FR-7	First-click safety	T-4
FR-8	Flood-fill cascade	T-5
FR-10	Chording precondition	T-6
FR-12	Shield blocks next mine reveal	T-7
FR-15	Server-side score recomputation	T-9
FR-16	Duplicate-submission rejection	T-10

Chapter 4

Game Design and Algorithms

This chapter describes the core Minesweeper game engine, implemented as a set of pure functions in `frontend/src/utils/minesweeper.js` and driven by the stateful hook `useMinesweeper.js`. Because the game engine is the technical core of this project, it is treated here as a dedicated chapter rather than as a subsection of the general implementation chapter.

4.1 Board Representation

The board is represented as a two-dimensional array of cell objects. Each cell has four fields:

- `isMine` – whether the cell contains a mine.
- `isRevealed` – whether the cell has been uncovered.
- `isFlagged` – whether the player has marked the cell.
- `adjacentMines` – the count of mines among the cell's up to 8 neighbours.

`createEmptyBoard` builds this grid with every cell initialised to safe, unrevealed, and unflagged, before any mines are placed.

4.2 Difficulty Configuration

Two difficulties are defined in a single configuration object, `DIFFICULTIES`, shown in Table 4.1. Centralising these values avoids duplicating grid dimensions across components.

Table 4.1: Supported difficulty configurations.

Difficulty	Grid	Mines	Safe Cells
Beginner	9×9	10	71
Advanced	16×16	40	216

4.3 First-Click Safety

Mines are not placed when the board is created; they are placed only in response to the player's first reveal, via `placeMines(board, mineCount, safeRow, safeCol)`. The function first builds a set containing the clicked cell and all of its neighbours (up to 8 cells), marking them as off-limits to mine placement. It then builds a list of every remaining candidate cell, shuffles it with a Fisher–Yates shuffle, and takes the first `mineCount` cells from the shuffled list as mine positions. Finally, it computes `adjacentMines` for every non-mine cell by counting mines among its neighbours. This guarantees that a player's first click, and the small neighbourhood around it, can never immediately end the game.

```
export function placeMines(board, mineCount, safeRow, safeCol) {
  const safeCells = new Set([`${safeRow},${safeCol}`]);
  getNeighbors(board, safeRow, safeCol).forEach((cell) => {
    safeCells.add(`${cell.row},${cell.col}`);
```

```
  });

  const candidates = [];
  for (let row = 0; row < board.length; row++) {
    for (let col = 0; col < board[0].length; col++) {
      if (!safeCells.has(`${row},${col}`)) candidates.push([row, col]);
    }
  }

  // Fisher-Yates shuffle
  for (let i = candidates.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [candidates[i], candidates[j]] = [candidates[j], candidates[i]];
  }

  const mineCells = candidates.slice(0, mineCount);
  // ... mark mineCells as isMine = true, then compute adjacentMines
}
```

Listing 4.1: Simplified first-click safety and mine placement (`minesweeper.js`).

4.4 Flood-Fill Reveal Algorithm

When a cell with `adjacentMines === 0` is revealed, all of its neighbours are safe to reveal automatically, and this cascades outward until every reachable zero-adjacency region and its numbered border has been uncovered. This is implemented in `revealCell(board, row, col)` using an explicit stack rather than recursion, which avoids call-stack depth concerns on larger boards such as the 16×16 Advanced grid. Each iteration pops a coordinate, skips it if already revealed or flagged, reveals it, and – if it has zero adjacent mines and is not a mine – pushes its unrevealed, unflagged neighbours onto the stack for processing.

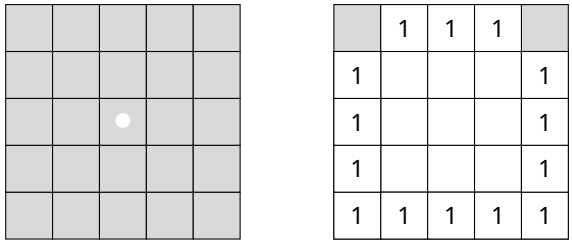
```
export function revealCell(board, row, col) {
  const newBoard = board.map((r) => r.map((cell) => ({ ...cell })));
  const stack = [[row, col]];

  while (stack.length > 0) {
    const [r, c] = stack.pop();
    const cell = newBoard[r][c];
    if (cell.isRevealed || cell.isFlagged) continue;
    cell.isRevealed = true;

    if (cell.adjacentMines === 0 && !cell.isMine) {
      getNeighbors(newBoard, r, c).forEach((neighbor) => {
        if (!neighbor.isRevealed && !neighbor.isFlagged) {
          stack.push([neighbor.row, neighbor.col]);
        }
      });
    }
  }
  return newBoard;
}
```

Listing 4.2: Iterative flood-fill reveal (`minesweeper.js`).

Figure 4.1 illustrates a worked example: revealing a single zero-adjacency cell cascades outward until it reaches cells with a non-zero mine count, which are revealed but do not propagate further.



Before: click on (2,2) **After: flood-fill cascade**

Figure 4.1: Worked example of the flood-fill reveal: clicking a zero-adjacency cell cascades to reveal the surrounding zero region and its numbered border, while unrelated cells remain hidden.

4.5 Flagging

`toggleFlag(board, row, col, mineCount)` toggles a cell's flag state. A revealed cell cannot be flagged. A new flag is only placed if the current number of flags on the board (`countFlags`) is below the mine count for the active difficulty, preventing the flag counter from going negative in the UI.

4.6 Chording

Chording lets the player reveal all unflagged neighbours of an already-revealed numbered cell in one action, once the surrounding flags match its number. `chordCell(board, row, col)` first checks that the target cell is revealed, has a non-zero `adjacentMines` value, and is not itself a mine. It then counts flagged neighbours; if that count does not equal `adjacentMines`, the board is returned unchanged. Otherwise, every unflagged, unrevealed neighbour is revealed via `revealCell` (triggering flood-fill where applicable). If a chord reveals a mine – i.e. the player mis-flagged – the calling code in `useMinesweeper` detects a revealed mine cell in the resulting board and ends the game.

```
export function chordCell(board, row, col) {
  const cell = board[row][col];
  if (!cell.isRevealed || cell.adjacentMines === 0 || cell.isMine) return board;

  const neighbors = getNeighbors(board, row, col);
  const adjacentFlagCount = neighbors.filter((n) => n.isFlagged).length;
  if (adjacentFlagCount !== cell.adjacentMines) return board;

  let workingBoard = board;
  neighbors
    .filter((n) => !n.isFlagged && !n.isRevealed)
    .forEach((n) => { workingBoard = revealCell(workingBoard, n.row, n.col); });
  return workingBoard;
}
```

Listing 4.3: Chording logic (`minesweeper.js`).

4.7 Hint System

`findHintCell(board)` collects every cell that is not a mine, not revealed, and not flagged, and returns one at random. The calling hook records the hint usage (capped at `MAX_HINTS = 3`), stores the returned coordinates as the currently highlighted `hintCell`, and clears

the highlight after `HINT_HIGHLIGHT_MS = 3000` milliseconds via a timeout. Each hint used subtracts `HINT_COST = 50` points from the eventual score (Section 4.11).

4.8 Shield Ability

Each game grants the player exactly one shield, tracked as a three-state value: `available`, `active`, and `used`. Activating the shield (`activateShield`) is only permitted while it is `available`. While `active`, the next time the player reveals a mine, `revealCellAt` sets the shield to `used` and returns without ending the game or revealing the mine, letting play continue. Figure 4.2 shows the shield's state machine. Note that because mines are placed only on the first click (Section 4.3), an active shield can, in principle, be consumed by the very first reveal if that reveal happens to land on a mine-designated cell outside the guaranteed-safe neighbourhood – though in practice first-click safety makes this impossible for the first click itself, and the shield instead protects the player's first mine encounter on any later click.

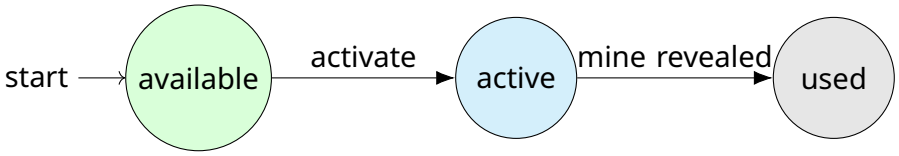


Figure 4.2: Shield ability state machine.

4.9 Game State Machine

The overall game progresses through three states, tracked as `gameStatus` in `useMinesweeper`: `ready` (board created but mines not yet placed), `playing` (first click made, timer running), and `lost` (a mine was revealed without an active shield, or the timer reached zero). There is deliberately no won/`completed-board` state.

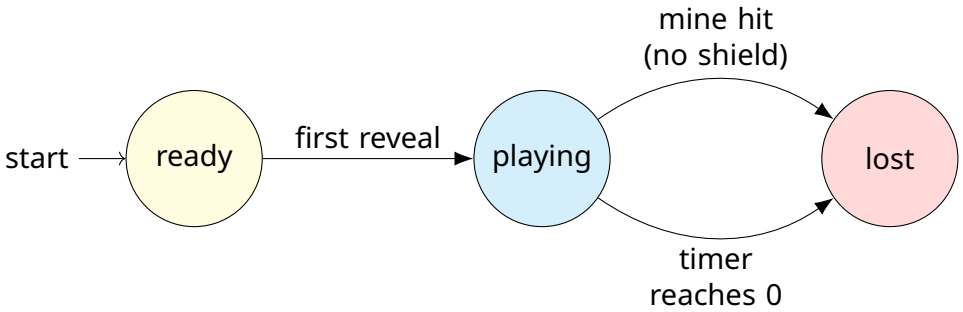


Figure 4.3: Game status state machine. There is no explicit "won" state; the game is a timed survival mode.

4.10 Survival Mode and the Absence of a Win Condition

Every game begins with a fixed countdown of `TIME_LIMIT_SECONDS = 120`. The game does not end when all safe cells are revealed; the player may continue revealing cells (subject to normal Minesweeper deduction) until either a mine is hit or the timer expires. This design choice is reflected in the database schema (Chapter 6), which stores

`cells_revealed` and `time_taken` but has no `result`/win-loss column – an explicit migration (`migration-remove-win-condition.sql`) removed a previous `result` column when survival mode was adopted.

4.11 Scoring Formula

The score for a completed game is:

$$\text{score} = \max(0,\ (\text{cellsRevealed} \times 10) + (\text{timeElapsed} \times 0) - (\text{hintsUsed} \times 50))$$

`POINTS_PER_SECOND` is deliberately set to `0`. Although the constant exists in the formula, it is defined as zero specifically to prevent a player from inflating their score simply by surviving passively (e.g. letting the timer run without revealing cells); the score is driven entirely by the number of safe cells actually revealed, less a fixed penalty per hint used. The same formula is evaluated twice: once on the client for immediate UI feedback (`calculateScore` in `minesweeper.js`), and independently on the server when the result is saved (Section 8.5), so that the value shown to the player during play is never the value actually trusted for the leaderboard.

```
export const POINTS_PER_CELL = 10;
export const POINTS_PER_SECOND = 0;
export const HINT_COST = 50;

export function calculateScore(cellsRevealed, timeElapsed, hintsUsed) {
  const rawScore = cellsRevealed * POINTS_PER_CELL
                 + timeElapsed * POINTS_PER_SECOND
                 - hintsUsed * HINT_COST;
  return Math.max(0, rawScore);
}
```

Listing 4.4: Client-side score formula (`minesweeper.js`).

4.12 Decision Flow of a Cell Reveal

Figure 4.4 summarises the decision logic executed each time the player clicks a cell, as implemented in `revealCellAt` within `useMinesweeper.js`.

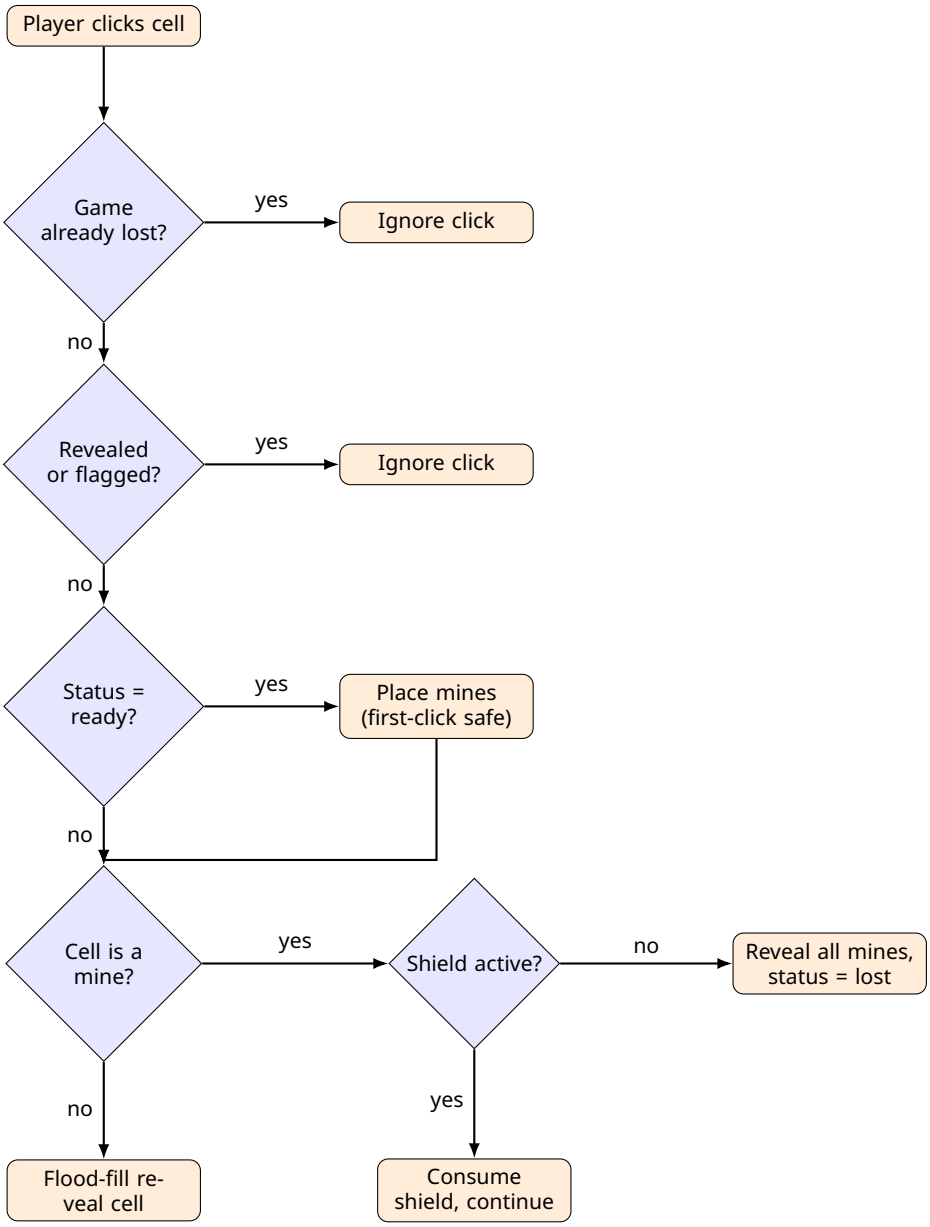


Figure 4.4: Decision flow for handling a cell reveal in `revealCellAt`.

Chapter 5

System Architecture and Software Design

5.1 Overall Architecture

The application follows a three-tier architecture, but unlike a classic server-rendered PHP application, the frontend is a React single-page application that communicates with the backend purely through a JSON API, rather than through full-page navigations. Figure 5.1 shows the overall structure.

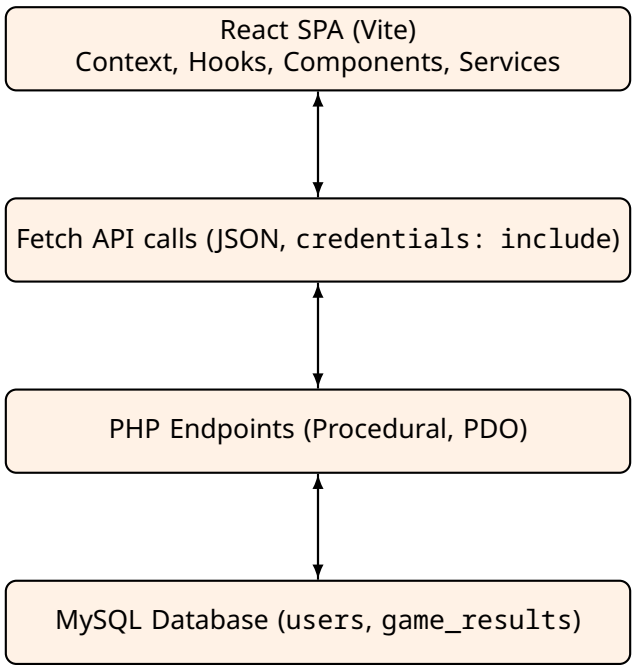


Figure 5.1: Overall three-tier architecture of the application.

5.2 Frontend Architecture

The frontend is organised into four layers:

- **Context providers** – `AuthContext` holds the current user and authentication status; `ThemeContext` holds the light/dark theme preference; a `ToastContext` manages transient notification messages.
- **Custom hooks** – `useMinesweeper` is the central state machine for a single game session, wrapping the pure functions from Chapter 4 with React state and effects (the countdown timer, hint highlight timeout, and automatic score submission).
- **Components / Pages** – page-level components (Login, Register, Dashboard, Game, Leaderboard, History) composed from smaller presentational components (the board grid, a cell, the stats cards).
- **Services** – `api.js` wraps the browser `fetch` API with a consistent JSON request/response contract; `authService.js` and `gameService.js` expose typed functions (e.g. `login`, `register`, `saveScore`, `getLeaderboard`) built on top of it.

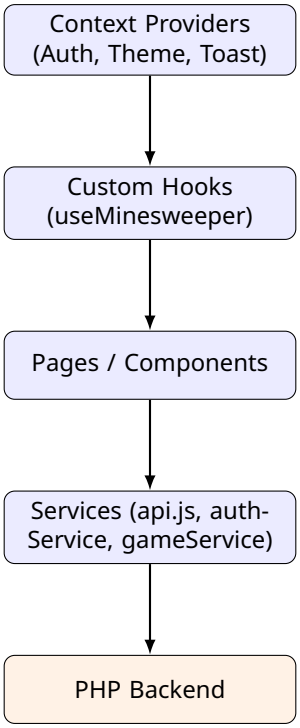


Figure 5.2: Frontend layered data flow.

5.3 Backend Architecture

The backend is written in procedural PHP, organised as one script per endpoint (e.g. `register.php`, `login.php`, `save-score.php`, `leaderboard.php`, `history.php`, `stats.php`), plus shared includes for the database connection and session/authentication helpers. Every protected endpoint begins by calling a shared `require_login()` helper, which checks the PHP session for a logged-in user ID and terminates the request with an error response if it is absent.

5.4 Client-Side Route Guarding vs. Server-Side Authorization

A React `ProtectedRoute` component wraps pages such as the dashboard and the game page, redirecting to the login page if `AuthContext` does not currently hold an authenticated user. It is important to note that this client-side check is a *user-experience* convenience only: it prevents an unauthenticated user from momentarily seeing a protected page's markup, but it grants no actual access to data. The real security boundary is enforced entirely server-side, by `require_login()` checking the PHP session on every protected endpoint (Section 8.2 in Chapter 8). A user could bypass `ProtectedRoute` entirely (e.g. by disabling JavaScript route checks) and would still be unable to retrieve another user's data, because the backend independently authenticates every request.

5.5 API Communication

All requests from the frontend to the backend are issued with `credentials: "include"`, so that the PHP session cookie is sent with cross-origin requests. Responses are JSON objects with a consistent shape (a success flag and either a data payload or an error message), which the `api.js` wrapper parses uniformly for every service call.

Chapter 6

Database Design

6.1 Database Overview

The application uses a MySQL relational database with two tables: `users` and `game_results`. The schema is intentionally small, reflecting the project's single-role, two-entity data model.

6.2 Tables

Table 6.1: `users` table.

Column	Type	Description
id	INT, PK, AUTO_INCREMENT	Unique user identifier
username	VARCHAR, UNIQUE	Display name, 3–50 characters
email	VARCHAR, UNIQUE	Login email address
password_hash	VARCHAR	Bcrypt password hash via `password_hash()`
created_at	DATETIME	Account creation timestamp

Table 6.2: `game_results` table.

Column	Type	Description
id	INT, PK, AUTO_INCREMENT	Unique game record identifier
user_id	INT, FK $\rightarrow$ users(id)	Owning player
difficulty	ENUM	`beginner` or `advanced`
score	INT	Server-recomputed final score
time_taken	INT	Seconds survived (0–120)
cells_revealed	INT	Number of safe cells revealed
hints_used	INT	Number of hints used (0–3)
created_at	DATETIME	Result submission timestamp

6.3 Relationships

The relationship between `users` and `game_results` is a standard one-to-many relationship: one user can accumulate many game records over time, so `user_id` is stored as a foreign key on the `game_results` side.

6.4 Important Constraints

- Foreign key: `game_results.user_id` references `users.id`.

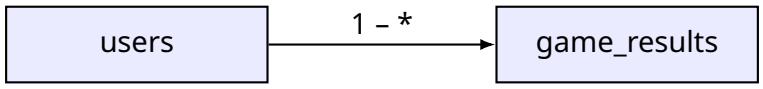


Figure 6.1: Entity relationship between `users` and `game_results`.

- Unique constraints on `users.username` and `users.email` prevent duplicate accounts.
- ENUM constraint on `game_results.difficulty` restricts the column to exactly `beginner` or `advanced`.
- An index is defined on `game_results.user_id` (and on `score` for leaderboard ordering) to support efficient history and ranking queries.

6.5 Schema Evolution

An early version of the schema included a `result` column recording a win/loss outcome, consistent with a classic "clear the board" win condition. When the game was redesigned as a timed survival mode (Section *Survival Mode and the Absence of a Win Condition*, Chapter 4), this column was no longer meaningful and was removed via `migration-remove-win-condition` since a completed game is now fully described by its difficulty, time survived, cells revealed, and hints used.

Chapter 7

Implementation

7.1 Frontend Implementation

The frontend is built with React and bundled with Vite, using plain JavaScript (no TypeScript) and component-scoped CSS. No external UI component framework is used for the game board itself; the board is rendered as a CSS grid of cell components, each responding to left-click (reveal), right-click (flag), and, on an already-revealed numbered cell, click-again (chord).

7.2 Backend Implementation

The backend is written in procedural PHP, without a framework, as a collection of standalone endpoint scripts plus shared includes for the database connection and authentication helpers. Unlike a typical `mysqli`-based implementation, database access uses PHP's PDO extension with prepared statements throughout, so that every query involving user-supplied data is parameterized.

7.3 API Endpoints

Table 7.1 summarises the backend endpoints exposed by the application.

Table 7.1: Summary of backend API endpoints.

Endpoint	Method	Auth	Purpose
register.php	POST	No	Create a new user account
login.php	POST	No	Authenticate and start a session
logout.php	POST	Yes	Destroy the current session
save-score.php	POST	Yes	Recompute and persist a completed game's result
leaderboard.php	GET	Yes	Return the top scores, optionally filtered by difficulty
history.php	GET	Yes	Return the logged-in user's past games
stats.php	GET	Yes	Return the logged-in user's aggregate statistics

7.4 Representative Frontend Code: API Wrapper

```
const API_BASE = import.meta.env.VITE_API_URL;

export async function apiRequest(endpoint, options = {}) {
  const response = await fetch(`${API_BASE}${endpoint}`, {
    credentials: "include",
```

```
    headers: { "Content-Type": "application/json" },
    ...options,
  });
  const data = await response.json();
  if (!response.ok || !data.success) {
    throw new Error(data.message || "Request failed");
  }
  return data;
}
```

Listing 7.1: Simplified fetch wrapper (`services/api.js`).

7.5 Representative Backend Code: Score Submission Endpoint

`save-score.php` is the most security-relevant endpoint in the system, since it is the only point at which client-reported gameplay metrics are converted into a persisted, leaderboard-visible score. Its validation logic is described fully in Section 8.5 of Chapter 8; a simplified outline is shown below.

```
<?php
require_once "includes/db.php";
require_once "includes/auth.php";
require_login();

$input = json_decode(file_get_contents("php://input"), true);
$difficulty = $input["difficulty"];
$timeTaken = (int) $input["time_taken"];
$cellsRevealed = (int) $input["cells_revealed"];
$hintsUsed = (int) $input["hints_used"];

// Bound-check against the known limits for this difficulty
validate_metrics($difficulty, $timeTaken, $cellsRevealed, $hintsUsed);

// Reject if an identical result was submitted in the last 10 seconds
reject_if_duplicate($userId, $difficulty, $timeTaken, $cellsRevealed, $hintsUsed);

// Recompute the score server-side; never trust a client-supplied score
$score = max(0, $cellsRevealed * 10 - $hintsUsed * 50);

$stmt = $pdo->prepare(
  "INSERT INTO game_results (user_id, difficulty, score, time_taken, cells_revealed
    , hints_used)
   VALUES (?, ?, ?, ?, ?, ?)"
);
$stmt->execute([$userId, $difficulty, $score, $timeTaken, $cellsRevealed,
   $hintsUsed]);
```

Listing 7.2: Simplified score submission endpoint (`save-score.php`).

7.6 Configuration Management

Difficulty parameters are defined once on the frontend in the `DIFFICULTIES` configuration object (grid size and mine count) and independently on the backend as the bounds used by `validate_metrics()` (maximum safe cells per difficulty). This duplication – rather than a single shared source of truth – is noted as a maintainability limitation in Chapter 12.

Chapter 8

Security

Security is treated as a dedicated chapter because the project's central integrity concern – preventing a client from fabricating a game result – is addressed through a specific set of server-side mechanisms that go beyond generic web-application hardening.

8.1 Password Storage

Passwords are hashed at registration time with `password_hash($password, PASSWORD_DEFAULT)` and verified at login time with `password_verify()`. Plaintext passwords are never stored or logged.

8.2 Session Security

Authentication is session-based, using PHP's native session mechanism.

- **Session fixation mitigation** – on successful login, the session ID is regenerated with `session_regenerate_id(true)`, preventing an attacker from fixing a victim's session ID before login.
- **Cookie hardening** – the session cookie is configured as HTTP-only (inaccessible to client-side JavaScript), with `SameSite=Lax` and scoped to the application's domain.
- **Server-side authorization** – every protected endpoint calls a shared `require_login()` helper that checks for a valid session before performing any work, independent of any client-side route guard (Chapter 5).

8.3 Cross-Origin Resource Sharing (CORS)

The backend restricts cross-origin requests to an explicit allow-list of known frontend origins, checked against the request's `Origin` header, rather than reflecting an arbitrary origin or using a wildcard. This is required because the frontend (Vite dev server / deployed SPA) and backend are served from different origins, and CORS must be configured correctly for the session cookie to be sent and accepted.

8.4 SQL Injection Prevention

All database access uses PDO prepared statements with bound parameters. No endpoint concatenates user-supplied input directly into a SQL string.

8.5 Server-Side Score Validation

Because the game engine runs entirely on the client (Chapter 4), the score value computed in the browser during play is treated as untrusted input. `save-score.php` recomputes the score independently from the raw submitted metrics (`cells_revealed`, `time_taken`, `hints_used`) using the same formula described in Section 4.11, and additionally bounds each metric against the known limits for the submitted difficulty (for

example, `cells_revealed` cannot exceed the total safe-cell count for that difficulty, and `hints_used` cannot exceed 3). A submission whose metrics fall outside these bounds is rejected. This means the value shown to the player at the end of a game is never the value actually written to the `game_results` table or shown on the leaderboard; the stored value is always the server's own computation.

8.6 Duplicate / Replay Submission Defense

To reduce the risk of the same completed game being submitted multiple times (whether accidentally or as a deliberate replay attempt), the backend computes a signature over the combination of user ID, difficulty, time taken, cells revealed, and hints used (an MD5 digest of the concatenated values), and rejects a new submission that matches a previous submission's signature within a 10-second window. This is a heuristic defense rather than a cryptographically strong nonce or idempotency-key mechanism; its scope and limitations are discussed further in Chapter 12.

8.7 Security Measures Summary

Table 8.1: Summary of applied security measures.

Measure	Purpose
Bcrypt password hashing	Prevent plaintext password exposure on data breach
Session ID regeneration on login	Mitigate session fixation attacks
HTTP-only, SameSite cookies	Reduce cookie theft via XSS / cross-site request risk
CORS origin allow-list	Prevent arbitrary origins from making authenticated requests
PDO prepared statements	Prevent SQL injection
Server-side score recomputation	Prevent client-fabricated leaderboard scores
Metric bounds checking	Prevent out-of-range or impossible gameplay metrics
Duplicate-submission signature	Reduce accidental/duplicate or naive replay submissions

8.8 Known Absences

For completeness, and in the same spirit of honest disclosure as the security chapter of the reference report, the following protections are *not* currently implemented: CSRF tokens on state-changing POST requests, request rate limiting, and enforced HTTPS in the development configuration (the session cookie's `secure` flag is `false` in the current setup). These are discussed further in Chapter 12.

Chapter 9

User Interface and Major Features

9.1 Page Walkthrough

9.1.1 Login and Registration

The login page (Figure 9.1) collects an email address and password. The registration page (Figure 9.2) collects a username, email, password, and password confirmation, and links back to the login page for existing users.

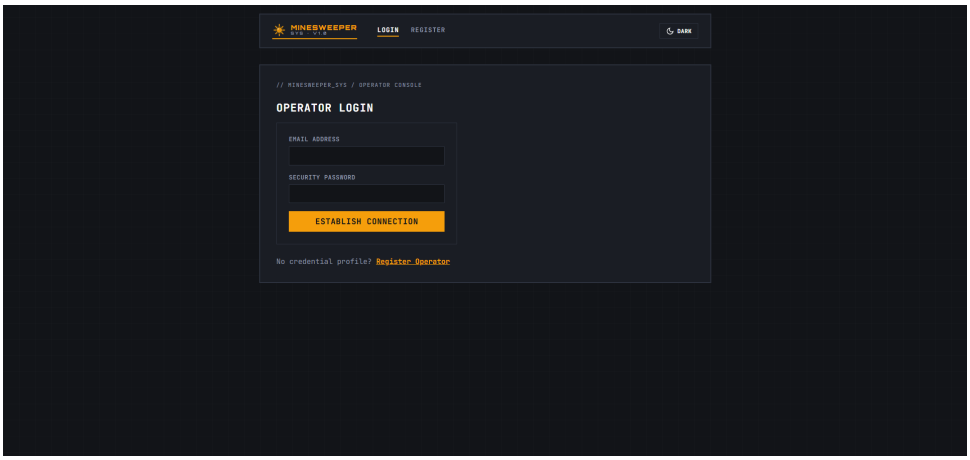


Figure 9.1: Login page (dark theme).

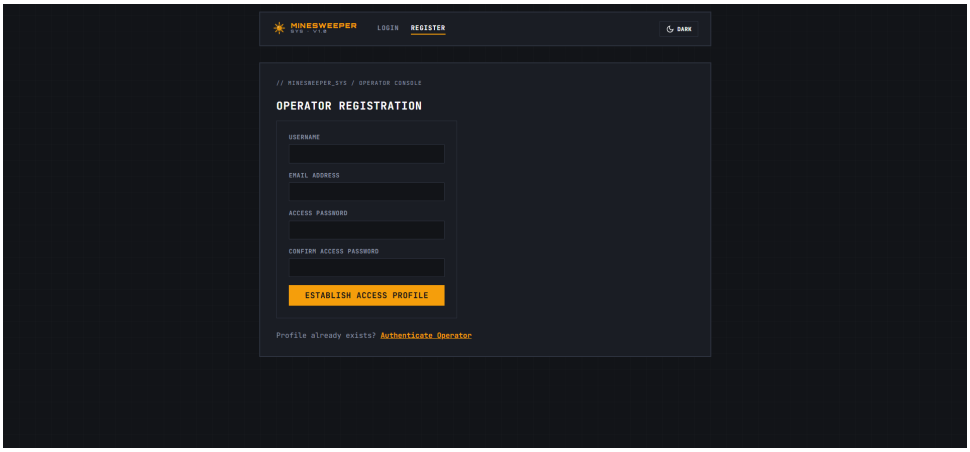


Figure 9.2: Registration page (dark theme).

9.1.2 Dashboard

The dashboard (Figures 9.3 and 9.4) is the player's home screen. It shows the player's identity, a set of aggregate statistics (games played, high score, average score, longest survival time, most cells revealed), filterable by "All Modes", "Beginner", or "Advanced", and quick-access links to the dashboard, game history, leaderboard, and a new game.

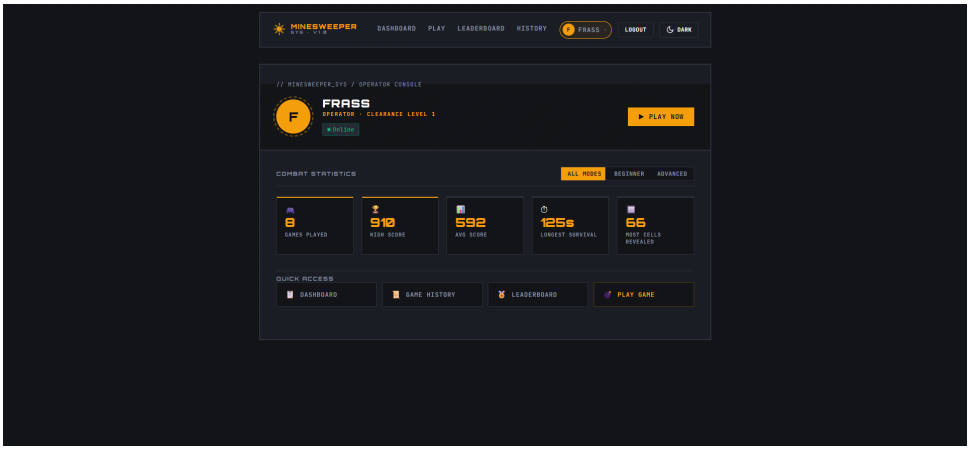


Figure 9.3: Player dashboard with combat statistics (dark theme).

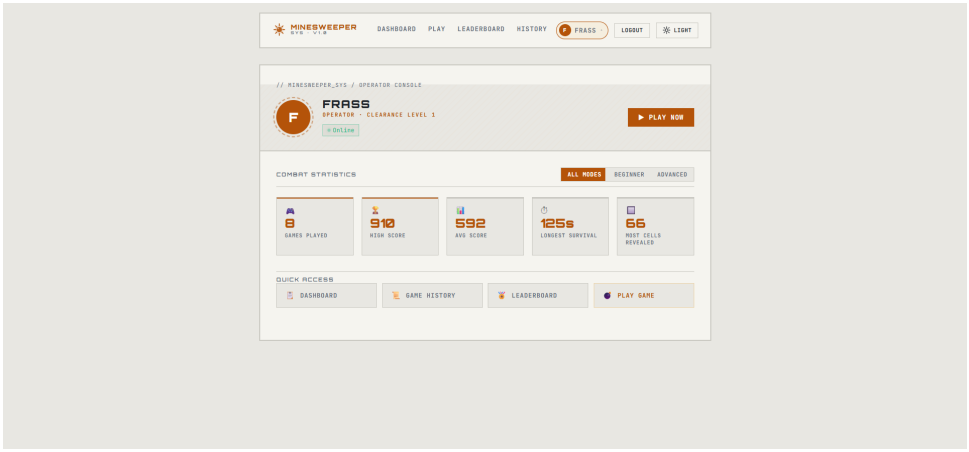


Figure 9.4: Player dashboard with combat statistics (light theme), showing theme support.

9.1.3 Game Page

The game page (Figure 9.5) shows a difficulty selector (Beginner / Advanced), the current mine count, flag count, and score, a countdown timer, the interactive board itself, a hint button showing remaining hints, a restart button, and the shield control with its current state (*Ready*, *Active*, or *Used*).

9.1.4 Leaderboard

The leaderboard page (Figure 9.6) lists the top scores across all players, filterable by "All", "Beginner", or "Advanced", with columns for rank, player, difficulty, score, time, cells revealed, and hints used.

9.1.5 History

The history page (Figure 9.7) lists the logged-in player's own past completed games, newest first, with the same metric columns as the leaderboard.

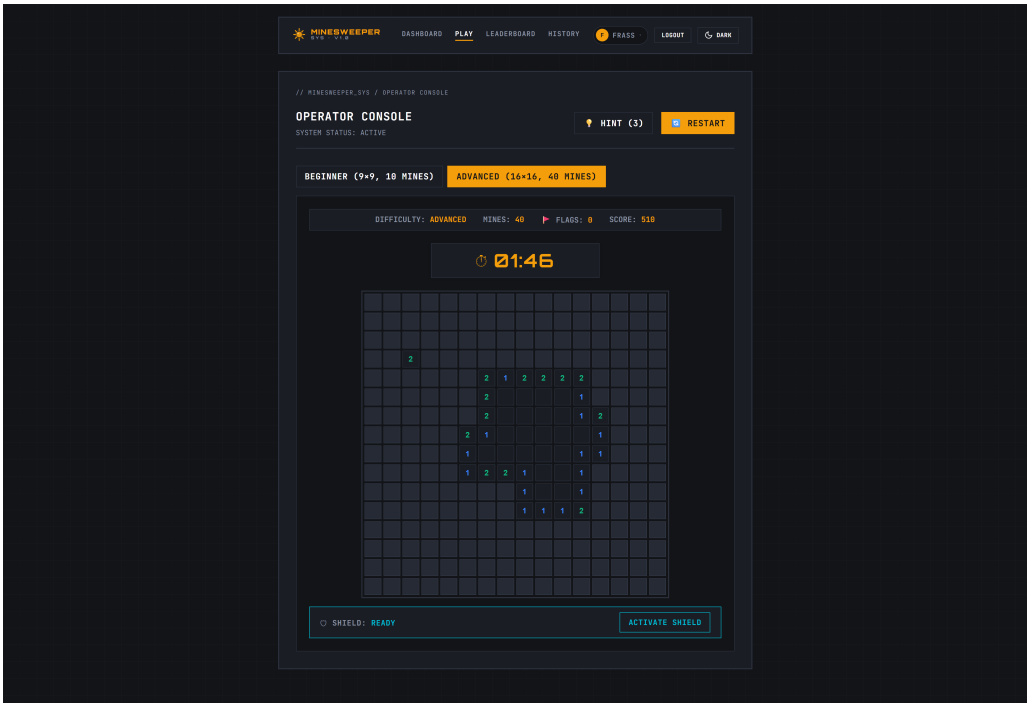


Figure 9.5: Active game on Advanced difficulty, showing the timer, score, flag count, partially revealed board, and shield control.

9.2 Theme Support

The application supports light and dark themes, toggled from the navigation bar and managed through `ThemeContext`. Figures 9.3 and 9.4 show the same dashboard rendered in both themes.

9.3 Feature Summary

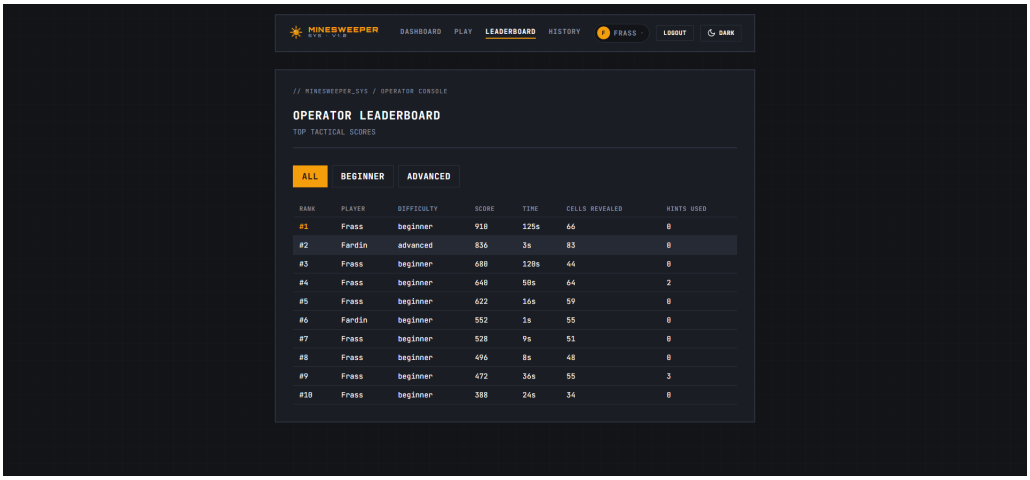


Figure 9.6: Global leaderboard, filtered to all difficulties.

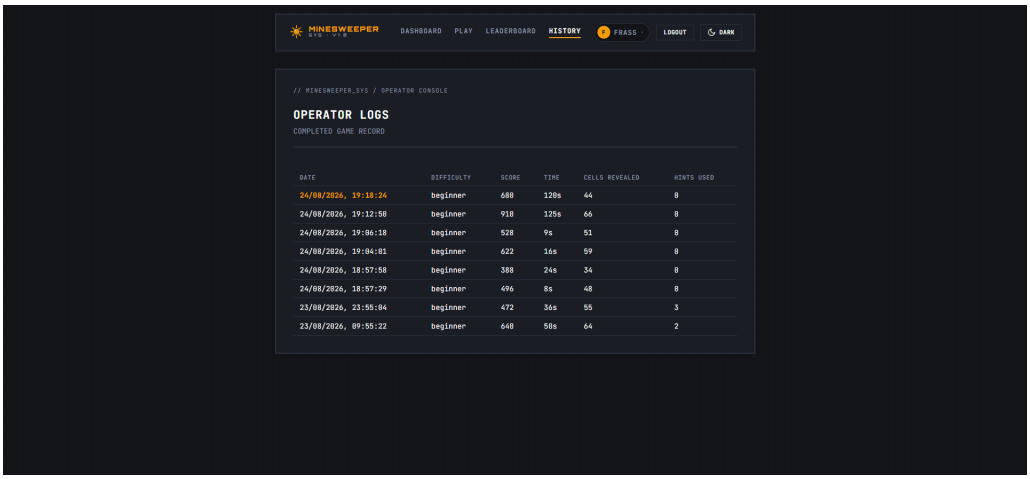


Figure 9.7: Player's personal game history.

Table 9.1: Core gameplay features.

Feature	**Description**
Two difficulties	Beginner (9×9, 10 mines) and Advanced (16×16, 40 mines)
First-click safety	Guaranteed-safe first reveal and its neighbourhood
Flood-fill reveal	Automatic cascading reveal of zero-adjacency regions
Flagging	Mark/unmark suspected mine cells, capped at the mine count
Chording	Reveal all unflagged neighbours of a satisfied numbered cell
Hints	Up to 3 random safe-cell highlights per game, at a score cost
Shield	One-time protection against the next mine reveal
Survival timer	120-second countdown; game ends on mine or timeout
Live scoring	Score recalculated and displayed during play

Table 9.2: Account and meta features.

Feature	Description
Registration / Login / Logout	Session-based account management
Personal dashboard	Aggregate statistics, filterable by difficulty
Game history	Full list of the player's own past completed games
Global leaderboard	Top scores across all players, filterable by difficulty
Theme support	Light/dark theme toggle

Chapter 10

Testing and Validation

10.1 Test Strategy

The system was validated using a structured, manual, black-box test plan, covering account management, the core game algorithms, server-side score validation, and cross-cutting security behaviours. Each test case specifies the steps to perform and the expected result, so the system's observable behaviour can be checked directly against the requirements in Chapter 3.

10.2 Test Cases

Table 10.1 lists the test cases referenced by the requirements-traceability table in Chapter 3, along with additional representative cases covering the game engine and security mechanisms.

10.3 Security Tests

Security-focused testing specifically targeted the mechanisms described in Chapter 8:

- **SQL injection resistance** (T-12) – confirmed that injection payloads in the login form fail cleanly, since all queries use PDO prepared statements.
- **Score tampering resistance** (T-9) – confirmed that metrics outside the valid range for a difficulty are rejected rather than trusted.
- **Duplicate submission resistance** (T-10) – confirmed that an identical result submitted twice in quick succession is rejected the second time.
- **Cross-user data isolation** (T-11) – confirmed that a user's history and statistics endpoints only ever return that user's own data, based on the server-side session rather than any client-supplied identifier.

10.4 Results

All test cases described above passed against the described behaviour of the system at the time of testing: first-click safety, flood-fill, and chording behave correctly across repeated trials; the shield correctly absorbs exactly one mine reveal per game; hints are capped at 3 and reduce the final score as expected; and the server-side score validation and duplicate-submission checks were both exercised successfully against manually crafted requests.

Table 10.1: Representative functional and security test cases.

ID	Steps	Expected Result
T-1	Register with a password under 6 characters.	Registration rejected with a validation error; no account created.
T-2	Inspect the stored password value after registration.	Value is a bcrypt hash, not the plaintext password.
T-3	Log in with the correct email but an incorrect password.	Login rejected; session not created.
T-4	Start a game and click any cell as the first move.	The clicked cell and all of its neighbours are guaranteed not to be mines.
T-5	Click a cell with zero adjacent mines.	All connected zero-adjacency cells, and their numbered border, are revealed in one action.
T-6	Flag the correct number of mines around a revealed numbered cell, then click it again.	All remaining unflagged neighbours are revealed at once (chord).
T-7	Activate the shield, then reveal a mine.	The game continues; the shield becomes *used*; the mine is not revealed as a loss.
T-8	Use all 3 hints in a single game.	The 4th hint request is not granted; each used hint reduces the final score by 50 points.
T-9	Submit a manually crafted request to `save-score.php` with an inflated `cells_revealed` value.	Request is rejected for exceeding the valid bound for the given difficulty.
T-10	Submit the same result twice within 10 seconds.	The second submission is rejected as a duplicate.
T-11	Log in as one user, then attempt to request another user's history via the API.	Only the authenticated user's own data is returned.
T-12	Attempt a classic SQL injection payload in the login form.	Login fails cleanly; no database error is exposed.
T-13	Let the timer reach zero without hitting a mine.	Game ends, all mines are revealed, and the result is submitted automatically.

Chapter 11

Results and Discussion

The completed application successfully implements a browser-playable Minesweeper game with persistent accounts, a global leaderboard, personal statistics, and game history, on top of a game engine that runs entirely on the client. In its current form, the system:

- Correctly guarantees a safe first click and its neighbourhood on every new game, regardless of difficulty.
- Correctly cascades reveals through zero-adjacency regions using an iterative, non-recursive flood-fill, avoiding recursion-depth concerns on the larger 16×16 Advanced board.
- Supports chording and a single-use shield ability as additional layers of player strategy beyond classic reveal/flag play.
- Enforces a strict separation between client-side gameplay and server-side score authority: the score shown to the player during play is never the value trusted for the leaderboard, which is instead recomputed and bounds-checked independently by `save-score.php`.
- Presents personal statistics, a full game history, and a global leaderboard, each filterable by difficulty, giving the player both short-term feedback and long-term progress tracking.

The decision to implement the game engine as a set of pure, testable functions (`minesweeper.js`), separate from the React state management in `useMinesweeper.js`, kept the core algorithms (first-click safety, flood-fill, chording, scoring) independent of the UI framework and straightforward to reason about and validate directly, as reflected in the test cases in Chapter 10. Likewise, treating the client-computed score as untrusted input and recomputing it server-side, rather than simply persisting whatever the client reports, directly addresses the score-integrity problem identified in the Introduction (Chapter 1).

The use of a single-page React frontend against a lightweight, framework-free procedural PHP API kept the backend small and easy to audit, while still demonstrating a complete client-server-database pipeline with real authentication, authorization, and data-integrity concerns, which suited the goals of the project.

Chapter 12

Limitations

- **No CSRF protection** – state-changing POST requests (e.g. `save-score.php`, `logout.php`) do not carry a CSRF token, relying instead on the CORS origin allow-list and SameSite cookie policy as the primary cross-site defenses.
- **Heuristic duplicate-submission signature** – the duplicate-submission check (Section 8.5) uses an MD5 digest of the submitted metrics and a 10-second window rather than a cryptographically strong, single-use nonce; a sufficiently patient or varied replay could still evade it.
- **Development-mode cookie configuration** – the session cookie's `secure` flag is currently `false`, appropriate for local HTTP development but not for a production HTTPS deployment.
- **Duplicated difficulty configuration** – the grid size and mine count for each difficulty are defined independently on the frontend (`DIFFICULTIES`) and as validation bounds on the backend (`validate_metrics()`), rather than from a single shared source of truth, so the two must be kept manually in sync.
- **No password reset flow** – a user who forgets their password has no self-service recovery mechanism.
- **Leaderboard scope** – the leaderboard displays a fixed top-10 list without pagination.
- **No rate limiting** – login and registration endpoints do not throttle repeated requests, leaving them potentially susceptible to brute-force attempts.
- **No formal foreign-key-backed role separation** – since there is only one user role, this is not currently a concern, but it is noted as a constraint on any future addition of privileged accounts.

Chapter 13

Future Enhancements

- **CSRF tokens** on all state-changing POST endpoints.
- **Stronger replay defense**, such as a server-issued, single-use game session token that must accompany a score submission, replacing the current MD5-signature heuristic.
- **Password reset** via a time-limited emailed reset link.
- **Single source of truth for difficulty configuration**, e.g. served from the backend (or database) and consumed by the frontend, instead of being duplicated in two places.
- **Additional difficulties or custom board sizes**, allowing players to configure grid dimensions and mine density directly.
- **Leaderboard pagination** beyond the current top-10 view.
- **Production-hardened session cookies**, enabling the `secure` flag once served over HTTPS.
- **Rate limiting** on authentication endpoints to mitigate brute-force login attempts.

These items are proposed improvements only; none of them are implemented in the current version of the application described in this report.

Chapter 14

Conclusion

This report has described the design and implementation of a web-based Minesweeper application built with a React/Vite frontend, a procedural PHP backend, and a MySQL database. The project's central technical contribution is the clear separation between a fully client-side game engine – covering first-click safety, an iterative flood-fill reveal, flagging, chording, hints, a shield ability, and a fixed-time survival mode – and a server that never trusts the client's own account of a completed game, instead independently re-computing and bounds-checking the score before it is persisted or shown on the leaderboard. Beyond gameplay, the system provides session-based authentication, a personal statistics dashboard, full game history, and a global leaderboard, all validated through a structured set of manual functional and security test cases. While the system has room to grow, particularly around CSRF protection, a stronger replay defense, and unifying the duplicated difficulty configuration, it fulfils its stated objective of pairing a responsive, client-driven Minesweeper experience with a server that remains the authoritative source of truth for every stored result.